МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«Сибирский государственный университет науки и технологий
имени академика М.Ф. Решетнева»**

Институт информатики и телекоммуникаций

Кафедра информатики и вычислительной техники

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

Алгоритмы и структуры данных

Лабораторная работа №2. Сравнительный анализ эффективности методов сортировки данных во внешней памяти.

Руководитель _____ В.В. Тынченко
подпись, дата инициалы, фамилия

Обучающийся БПИ24-02, 24151430 _____ В.Т. Гордов
номер группы, зачетной книжки подпись, дата инициалы, фамилия

Красноярск 2025 г.

ЦЕЛЬ РАБОТЫ

Изучение алгоритмов сортировки данных во внешней памяти, особенностей их программной реализации и эффективности работы на различных наборах исходных данных.

ПОСТАНОВКА ЗАДАЧИ

Разработать и реализовать программу, предназначенную для исследования времени работы и поведения двух выбранных методов внешней сортировки. При выборе методов, подлежащих реализации, необходимо ознакомиться с принципом оценивания результатов работы, изложенным ниже.

Провести исследование параметров работы, указанных в варианте задания методов сортировки на различных наборах данных. Представить отчет, содержащий результаты исследования и полученные выводы.

Оценка «Хорошо».

Программно реализовать следующие методы сортировки данных во внешней памяти: сортировка прямым слиянием; сортировка сбалансированным многопутевым слиянием.

Оценить быстродействие указанных методов и степень естественности их поведения.

ХОД РАБОТЫ

В ходе лабораторной работы была разработана программа, предназначенная для исследования времени работы и поведения методов сортировки данных во внешней памяти: сортировка прямым слиянием; сортировка сбалансированным многопутевым слиянием.

Была реализована структура для хранения статистики сортировки (Рисунок 1).

```
// Структура для хранения статистики сортировки
You, 18 часов назад | 1 author (You)
struct SortStats {
    long long comparisons = 0; // Количество сравнений
    long long swaps = 0; // Количество перестановок
    double time_ms = 0.0; // Время выполнения в миллисекундах
};
```

Рисунок 1 — Структура для хранения статистики сортировки

Была реализована функция для создания файла с данными (Рисунки 2 — 3). Пользователь выбирает размер последовательности, диапазон значений элементов последовательности и режим генерации — случайные значения, упорядоченная последовательность или упорядоченная в обратном порядке последовательность.

```
// Перечисление для выбора режима генерации данных
enum DataGenerationMode {
    RANDOM, // Случайные значения
    SORTED, // Отсортированная последовательность
    REVERSE_SORTED, // Последовательность в обратном порядке
};

/// @brief Генерации файла с данными для сортировки
/// @param filename имя файла для записи
/// @param size количество элементов
/// @param min_val минимальное значение элемента
/// @param max_val максимальное значение элемента
/// @param mode режим генерации данных
void generateDataFile(const string& filename, int size, int min_val, int max_val, DataGenerationMode mode) {
    ofstream file(filename);
    if (!file) {
        cerr << "ERROR: Failed to create file for writing data." << endl;
        return;
    }

    vector<int> data(size);

    switch (mode) // Генерируем случайные данные в зависимости от режима
    {
    case RANDOM:
        {
            random_device rd;
            mt19937 gen(rd());
            uniform_int_distribution<> distrib(min_val, max_val);
            for (int i = 0; i < size; ++i) {
                data[i] = distrib(gen);
            }
            break;
        }
    case SORTED:
        {
            for (int i = 0; i < size; ++i) {
                data[i] = i + min_val;
            }
            break;
        }
    case REVERSE_SORTED:
```

Рисунок 2 — Функция для создания файла с данными — часть 1

```
    case REVERSE_SORTED:
        {
            for (int i = 0; i < size; ++i) {
                data[i] = i + min_val;
            }
            reverse(data.begin(), data.end());
            break;
        }
    default:
        break;
    }

    // Записываем данные в файл в строчку через пробел
    for (int i = 0; i < size; ++i) {
        file << data[i] << (i == size - 1 ? "" : " "); // Не добавляем пробел после последнего элемента
    }
    file.close();
}
```

Рисунок 3 — Функция для создания файла с данными — часть 2

Была реализована функция прямой сортировки слиянием (Рисунки 4-6).

```
/// @brief Функция прямой сортировки слиянием
/// @param filename имя исходного файла с данными
/// @return Структура со статистикой сортировки
SortStats directMergeSort(const string& filename) {
    // Засекаем время начала сортировки
    auto start_time = chrono::high_resolution_clock::now();
    SortStats stats;

    // Открываем поток для чтения из файла
    ifstream initial_file_counter(filename);
    if (!initial_file_counter) {
        cerr << "ERROR: Failed to open the source file." << endl;
        return stats;
    }

    // Подсчитываем общее количество элементов в файле

    /// Счётчик элементов в файле
    int n = 0;
    int temp;
    while (initial_file_counter >> temp) {
        n++;
    }
    initial_file_counter.close();

    // Если элементов 1 или 0, сортировка не требуется
    if (n <= 1) {
        auto end_time = chrono::high_resolution_clock::now();
        stats.time_ms = chrono::duration<double, milli>(end_time - start_time).count();
        return stats;
    }

    // Используем три файла для сортировки
    string fileA = filename; // Основной файл с данными
    string fileB = "temp_b.txt"; // Временный файл для разделения и слияния
    string fileC = "temp_c.txt"; // Временный файл для разделения и слияния

    // Цикл сортировки — увеличиваем размер серий в 2 раза на каждой итерации
    for (int run_size = 1; run_size < n; run_size *= 2) {
        ifstream source(fileA); // Поток для чтения из основного файла
        ofstream dest_b(fileB); // Поток для записи во временный файл
        ofstream dest_c(fileC); // Поток для записи во временный файл

        // Разделяем элементы из основного файла и поочерёдно записываем в два временных файла
        int current_val; // Переменная для записи/чтения значений
        int count = 0; // Счётчик текущей серии цикла
        while (source >> current_val) {
            // Определяем файл для записи по номеру текущей серии
            if ((count / run_size) % 2 == 0) {
                dest_b << current_val << " ";
            } else {
                dest_c << current_val << " ";
            }
            stats.swaps++; // Увеличиваем счетчик перестановок
            count++;
        }
        source.close();
        dest_b.close();
        dest_c.close();
```

Рисунок 4 — Функция прямой сортировки слиянием — часть 1

```
source.close();
dest_b.close();
dest_c.close();

// Теперь сливаем отсортированные серии из файлов B и C обратно в файл A
ofstream dest_a(fileA); // Поток для записи в основной файл
ifstream source_b(fileB); // Поток для чтения из временного файла
ifstream source_c(fileC); // Поток для чтения из временного файла

// Читаем первые элементы из каждого временного файла
int val_b, val_c;
bool has_b = static_cast<bool>(source_b >> val_b); // Есть ли элемент в файле B?
bool has_c = static_cast<bool>(source_c >> val_c); // Есть ли элемент в файле C?

// Цикл слияния серий
while(has_b || has_c) {
    int b_count = 0; // Счётчик элементов временного файла в текущей серии
    int c_count = 0; // Счётчик элементов временного файла в текущей серии

    // Сливаем элементы из текущих серий, пока не закончится одна из них
    // или пока не будет обработано run_size элементов из каждой серии
    while(b_count < run_size && c_count < run_size && has_b && has_c) {
        stats.comparisons++; // Увеличиваем счетчик сравнений

        // Сравниваем текущие элементы и записываем меньший
        if (val_b <= val_c) {
            dest_a << val_b << " ";
            stats.swaps++; // Увеличиваем счетчик перестановок
            b_count++;

            has_b = static_cast<bool>(source_b >> val_b);
        } else {
            dest_a << val_c << " ";
            stats.swaps++; // Увеличиваем счетчик перестановок
            c_count++;

            has_c = static_cast<bool>(source_c >> val_c);
        }
    }

    // Дописываем оставшиеся элементы из серии B (если серия C закончилась)
    while(b_count < run_size && has_b) {
        dest_a << val_b << " ";
        stats.swaps++; // Увеличиваем счетчик перестановок
        b_count++;
        has_b = static_cast<bool>(source_b >> val_b);
    }
    // Дописываем оставшиеся элементы из серии C (если серия B закончилась)
    while(c_count < run_size && has_c) {
        dest_a << val_c << " ";
        stats.swaps++; // Увеличиваем счетчик перестановок
        c_count++;
        has_c = static_cast<bool>(source_c >> val_c);
    }
}

// Закрываем все файлы
```

Рисунок 5 — Функция прямой сортировки слиянием — часть 2

```
        // Закрываем все файлы
        dest_a.close();
        source_b.close();
        source_c.close();
    }

    // Удаляем все временные файлы
    remove(fileB.c_str());
    remove(fileC.c_str());

    // Засекаем время окончания и вычисляем общее время сортировки
    auto end_time = chrono::high_resolution_clock::now(); // Время окончания сортировки
    stats.time_ms = chrono::duration<double, milli>(end_time - start_time).count();

    return stats;
}
```

Рисунок 6 — Функция прямой сортировки слиянием — часть 3

Была реализована функция сбалансированной многопутевой сортировки (Рисунки 7-9).

```
/// @brief Функция сбалансированной многопутевой сортировки слиянием
/// @param filename имя исходного файла с данными
/// @param m количество путей (временных файлов) для сортировки
/// @return Структура со статистикой сортировки
SortStats balancedMultiwayMergeSort(const string& filename, int m) {
    // Проверка корректности параметра m (количество путей)
    if (m < 2) {
        cerr << "ERROR: Number of ways must be >2." << endl;
        return {};
    }

    // Засекаем время начала сортировки
    auto start_time = chrono::high_resolution_clock::now();
    SortStats stats;

    // Открываем поток для чтения из файла
    ifstream initial_file_counter(filename);
    if (!initial_file_counter) {
        cerr << "ERROR: Failed to open the source file." << endl;
        return stats;
    }

    // Подсчитываем общее количество элементов в файле

    /// Счётчик элементов в файле
    int n = 0;
    int temp;
    while (initial_file_counter >> temp) {
        n++;
    }
    initial_file_counter.close();

    // Если элементов 1 или 0, сортировка не требуется
    if (n <= 1) {
        auto end_time = chrono::high_resolution_clock::now();
        stats.time_ms = chrono::duration<double, milli>(end_time - start_time).count();
        return stats;
    }

    /// Массивы названий временных файлов, используются поочерёдно для чтения и записи
    vector<string> files1, files2;
    for (int i = 0; i < m; ++i) {
        files1.push_back("temp_1_" + to_string(i) + ".txt");
        files2.push_back("temp_2_" + to_string(i) + ".txt");
    }

    // Создаём начальные серии и распределяем исходные данные по m файлам поочерёдно

    ifstream source(filename);
    vector<ofstream> dests;
    for (int i = 0; i < m; ++i) {
        dests.emplace_back(files1[i]);
    }
    int file_index = 0;
    while (source >> temp) {
        dests[file_index] << temp << " ";
        stats.swaps++; // Увеличиваем счетчик перестановок
        file_index = (file_index + 1) % m; // Переход к следующему файлу
    }
```

Рисунок 7 — Функция сбалансированной многопутевой сортировки — часть 1

```
    }

    // Инициализируем размер серий и указатели на используемые файлы
    long long run_size = 1; // Размер текущих серий
    vector<string>* from_files = &files1; // Файлы, из которых читаем
    vector<string>* to_files = &files2; // Файлы, в которые записываем

    // Основной цикл сортировки - продолжаем, пока размер серий меньше общего количества элементов
    while (run_size < n) {
        vector<ifstream> sources; // Массив потоков файлов для чтения
        for (int i = 0; i < m; ++i) {
            sources.emplace_back((*from_files)[i]);
        }

        vector<ofstream> dests; // Массив потоков файлов для записи
        for (int i = 0; i < m; ++i) {
            dests.emplace_back((*to_files)[i]);
        }

        int dest_index = 0; // Индекс файла назначения для записи

        // Цикл объединения серий
        while (true) {
            // Очередь с приоритетом для нахождения минимального элемента среди всех файлов
            priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;

            // Массив для отслеживания количества элементов в текущей серии
            vector<int> current_run_counts(m, 0);

            // Читаем первый элемент из каждого файла в очередь
            for (int i = 0; i < m; ++i) {
                if (sources[i] >> temp) {
                    pq.push({temp, i}); // Пары {значение, индекс файла}
                }
            }

            // Если очередь пуста, все элементы обработаны
            if (pq.empty()) {
                break;
            }

            // Извлекаем элементы из очереди в отсортированном порядке
            while (!pq.empty()) {
                pair<int, int> min_pair = pq.top();
                pq.pop();

                int value = min_pair.first; // Значение минимального элемента
                int file_idx = min_pair.second; // Индекс файла, из которого взят элемент

                // Записываем минимальный элемент в текущий файл назначения
                dests[dest_index] << value << " ";
                stats.swaps++; // Увеличиваем счетчик перестановок

                // Увеличиваем счетчик элементов в текущей серии для этого файла
                current_run_counts[file_idx]++;
```

Рисунок 8 — Функция сбалансированной многопутевой сортировки — часть 2

```
                current_run_counts[file_idx]++;

                // Если текущая серия еще не закончилась, читаем следующий элемент из этого файла
                if (current_run_counts[file_idx] < run_size) {
                    if (sources[file_idx] >> temp) {
                        stats.comparisons++; // Увеличиваем счетчик сравнений
                        pq.push({temp, file_idx});
                    }
                }
            }
            // Переходим к следующему файлу назначения для равномерного распределения
            dest_index = (dest_index + 1) % m;
        }

        // Увеличиваем размер серий в m раз
        run_size *= m;
        // Меняем местами файлы чтения и записи для следующей итерации
        swap(from_files, to_files);
    }

    // Копируем отсортированные данные обратно в исходный файл
    // Результат находится в первом файле последнего набора
    {
        ofstream final_dest(filename);
        ifstream final_source((*from_files)[0]);
        while (final_source >> temp) {
            final_dest << temp << " ";
        }
    }

    // Удаляем все временные файлы
    for (int i = 0; i < m; ++i) {
        remove(files1[i].c_str());
        remove(files2[i].c_str());
    }

    // Засекаем время окончания и вычисляем общее время сортировки
    auto end_time = chrono::high_resolution_clock::now(); // Время окончания сортировки
    stats.time_ms = chrono::duration<double, milli>(end_time - start_time).count();

    return stats;
}
```

Рисунок 9 — Функция сбалансированной многопутевой сортировки — часть 3

Было реализовано консольное меню, позволяющее выбирать режимы работы программы (Рисунки 10-11).

```
int main() {
    int mode;
    do {
        cout << "\n===== Laboratory Word #2 =====\n";
        cout << "Goal: Comparative analysis of external memory sorting algorithmes efficiency.\n";
        cout << "Level 'Good': Direct merge sorting, balanced multiway merge sorting.\n\n";

        cout << "============= MENU =============\n";
        cout << "1. Sort a single data file\n";
        cout << "2. Statistical data collection mode\n";
        cout << "3. Close the program\n";
        cout << "Choose execution mode: ";

        cin >> mode;

        switch (mode) {
            case 1:
                singleSortMode();
                break;
            case 2:
                statisticsMode();
                break;
            case 3:
                cout << "Exiting the program..." << endl;
                break;
            default:
                cout << "ERROR: Invalid choice." << endl;
        }

    } while (mode != 3);

    return 0;
}
```

Рисунок 10 — Код меню

```
===== Laboratory Word #2 =====
Goal: Comparative analysis of external memory sorting algorithmes efficiency.
Level 'Good': Direct merge sorting, balanced multiway merge sorting.

============= MENU =============
1. Sort a single data file
2. Statistical data collection mode
3. Close the program
Choose execution mode:
```

Рисунок 11 — Меню

Метод сортировки одного файла с данными (Рисунки 12-15) реализован следующим образом: пользователь вводит желаемые размер последовательности и диапазон значений элементов (от наименьшего до наибольшего возможных). По заданным параметрам случайным образом генерируется файл с данным. Далее пользователь выбирает желаемый метод внешней сортировки. Если выбрана сортировка сбалансированным многопутевым слияниям, то пользователь дополнительно указывает количество путей для слияния. Далее происходит сортировка сгенерированной последовательности в файле, по завершении которого в

консоль выводится статистика (время сортировки, количество сравнений и количество перестановок).

```
/// @brief Выполнение сортировки в одиночном режиме
void singleSortMode() {        You, 20 часов назад via  PR #35 • 2 лаба, работающая версия
    int size, min_val, max_val, method;
    string filename = "data.txt";

    // Получаем параметры последовательности от пользователя
    cout << "Enter the size of the numeric sequence: ";
    cin >> size;
    cout << "Enter the minimum value of an element: ";
    cin >> min_val;
    cout << "Enter the maximum value of an element: ";
    cin >> max_val;

    // Генерируем файл со случайными данными
    generateDataFile(filename, size, min_val, max_val, RANDOM);
    cout << "File \"" << filename << "\" with random data has been generated." << endl;

    // Пользователь выбирает метод сортировки
    cout << "\nChoose the external sorting method:\n";
    cout << "1. Direct merge sort\n";
    cout << "2. Balanced multiway merge sort\n";
    cout << "Your choice: ";
    cin >> method;

    SortStats stats;
    if (method == 1) {
        // Выполняем прямую сортировку слиянием
        stats = directMergeSort(filename);
    } else if (method == 2) {
        // Выполняем многопутевую сортировку слиянием
        int m; // Количество путей для сортировки
        cout << "Enter the number of ways (m) for merging (e.g., 3): ";
        cin >> m;
        stats = balancedMultiwayMergeSort(filename, m);
    } else {
        cout << "ERROR: Invalid choice." << endl;
        return;
    }

    // Выводим результаты сортировки
    cout << "\nSorting results:\n";
    cout << "Execution time: " << stats.time_ms << " ms\n";
    cout << "Number of comparisons: " << stats.comparisons << "\n";
    cout << "Number of swaps (writes): " << stats.swaps << "\n";
    cout << "Sorted data is in the file \"" << filename << "\"." << endl;
}
```

Рисунок 12 — Метод сортировки одного файла с данными

```
============= MENU =============
1. Sort a single data file
2. Statistical data collection mode
3. Close the program
Choose execution mode: 1
Enter the size of the numeric sequence: 1000
Enter the minimum value of an element: 1
Enter the maximum value of an element: 10000
File "data.txt" with random data has been generated.

Choose the external sorting method:
1. Direct merge sort
2. Balanced multiway merge sort
Your choice: 1

Sorting results:
Execution time: 372.654 ms
Number of comparisons: 8737
Number of swaps (writes): 20000
Sorted data is in the file "data.txt".
```

Рисунок 13 — Демонстрация работы метода сортировки одного файла с данными

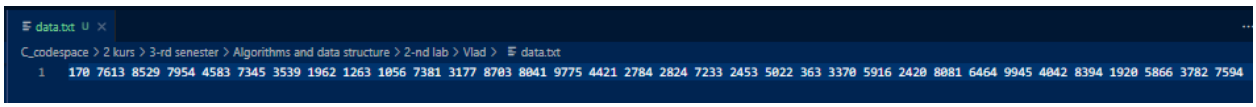


Рисунок 14 — Последовательность данных до сортировки

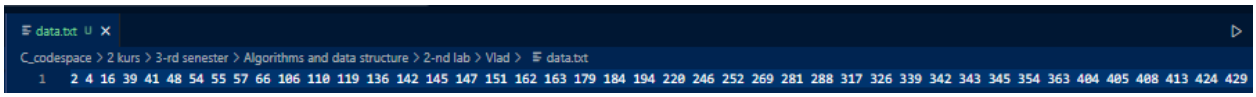


Рисунок 15 — Последовательность данных после сортировки

Метод накопления статистических данных (Рисунки 16-19) реализован следующим образом: пользователь выбирает желаемый способ формирования элементов последовательности (случайные значения, упорядоченная последовательность, обратно упорядоченная последовательность), диапазон размеров последовательности (от наименьшего к наибольшему возможным), шаг изменения размера и метод сортировки. После этого программа открывает цикл по указанным параметрам, в котором она создаёт последовательности, выполняет их сортировку и выводит статистические данные в отдельный файл.

```
/// @brief Выполнение сортировки в режима сбора статистики
void statisticsMode() {
    int data_mode_choice, sort_method_choice;
    int start_size, end_size, step;
    string output_filename;

    // Выбор режима генерации данных
    cout << "Choose the method of forming the sequence elements:\n";
    cout << "1. Random values\n";
    cout << "2. Sorted sequence\n";
    cout << "3. Values in reverse order\n";
    cout << "Your choice: ";
    cin >> data_mode_choice;

    // Преобразуем выбор пользователя в enum
    DataGenerationMode data_mode;
    switch (data_mode_choice) {
        case 1: data_mode = RANDOM; break;
        case 2: data_mode = SORTED; break;
        case 3: data_mode = REVERSE_SORTED; break;
        default: cout << "ERROR: Invalid choice.\n"; return;
    }

    // Получаем диапазон и шаг изменения размера данных
    cout << "Set the range of sequence size change:\n";
    cout << "Start size: ";
    cin >> start_size;
    cout << "End size: ";
    cin >> end_size;
    cout << "Step size: ";
    cin >> step;

    // Выбор метода сортировки
    cout << "\nChoose the external sorting method:\n";
    cout << "1. Direct merge sort\n";
    cout << "2. Balanced multiway merge sort\n";
    cout << "Your choice: ";
    cin >> sort_method_choice;

    int m;
    if(sort_method_choice == 2) {
        cout << "Enter the number of ways (m) for merging (e.g., 3): ";
        cin >> m;
    }

    // Получаем имя файла для сохранения статистики
    cout << "Enter the filename to save results (e.g., stats.csv): ";
    cin >> output_filename;

    // Открываем файл для записи статистики
    ofstream stats_file(output_filename);
    if (!stats_file) {
        cerr << "Failed to create statistics file." << endl;
        return;
    }

    // Записываем заголовок CSV файла
    stats_file << "Size Time_(ms)\n";
    // Устанавливаем формат вывода чисел
    stats_file << fixed << setprecision(0);
    cout << "\nStarting statistics collection...\n";

    // Цикл по различным размерам данных
    for (int size = start_size; size <= end_size; size += step) {
        cout << "Sorting for size " << size << "..." << endl;
        string temp_filename = "temp_data.txt";
        // Генерируем временный файл с данными в выбранном режиме
        generateDataFile(temp_filename, size, 0, size * 10, data_mode);

        SortStats stats;
        if (sort_method_choice == 1) {
            // Выполняем прямую сортировку слиянием
```

Рисунок 16 — Метод накопления статистических данных — часть 1

```
        SortStats stats;
        if (sort_method_choice == 1) {
            // Выполняем прямую сортировку слиянием
            stats = directMergeSort(temp_filename);
        } else if (sort_method_choice == 2) {
            // Выполняем многопутевую сортировку слиянием
            stats = balancedMultiwayMergeSort(temp_filename, m);
        } else {
            cout << "Invalid sorting method choice." << endl;
            return;
        }

        // Записываем результаты в файл статистики
        stats_file << size << " " << stats.time_ms << "\n";
    }

    stats_file.close();
    cout << "\nStatistics collection finished. Results saved to file \"" << output_filename << "\"." << endl;
}
```

Рисунок 17 — Метод накопления статистических данных — часть 2

```
===== Laboratory Word #2 =====
Goal: Comparative analysis of external memory sorting algorithmes efficiency.
Level 'Good': Direct merge sorting, balanced multiway merge sorting.

============= MENU =============
1. Sort a single data file
2. Statistical data collection mode
3. Close the program
Choose execution mode: 2
Choose the method of forming the sequence elements:
1. Random values
2. Sorted sequence
3. Values in reverse order
Your choice: 1
Set the range of sequence size change:
Start size: 1000
End size: 10000
Step size: 1000

Choose the external sorting method:
1. Direct merge sort
2. Balanced multiway merge sort
Your choice: 1
Enter the filename to save results (e.g., stats.csv): stats.csv

Starting statistics collection...
Sorting for size 1000...
Sorting for size 2000...
Sorting for size 3000...
Sorting for size 4000...
Sorting for size 5000...
Sorting for size 6000...
Sorting for size 7000...
Sorting for size 8000...
Sorting for size 9000...
Sorting for size 10000...

Statistics collection finished. Results saved to file "stats.csv".
```

Рисунок 18 — Демонстрация работы метода накопления статистических данных

```
stats.csv U ×
C_codespace > 2 kurs > 3-rd senester > Algorithms and data structure > 2-nd lab > Vlad > stats.csv > data
1   Size Time_(ms)
2   1000 393
3   2000 400
4   3000 642
5   4000 379
6   5000 413
7   6000 477
8   7000 526
9   8000 476
10  9000 529
11  10000 523
12
```

Рисунок 19 — Запись результатов метода накопления статистических данных в файл

Для тестирования первого режима программы были выбраны параметры: размер последовательности — 15000, диапазон значений элементов — от 1 до 10000.

Результаты сортировки прямым слиянием (Рисунок 20):

- Время — 522.426 мс.
- Количество сравнений — 190171.
- Количество перестановок — 420000.

```
===== Laboratory Word #2 =====
Goal: Comparative analysis of external memory sorting algorithmes efficiency.
Level 'Good': Direct merge sorting, balanced multiway merge sorting.

============= MENU =============
1. Sort a single data file
2. Statistical data collection mode
3. Close the program
Choose execution mode: 1
Enter the size of the numeric sequence: 15000
Enter the minimum value of an element: 1
Enter the maximum value of an element: 10000
File "data.txt" with random data has been generated.

Choose the external sorting method:
1. Direct merge sort
2. Balanced multiway merge sort
Your choice: 1

Sorting results:
Execution time: 522.426 ms
Number of comparisons: 190171
Number of swaps (writes): 420000
Sorted data is in the file "data.txt".
```

Рисунок 20 — Режим 1. Сортировка прямым слиянием

Результаты сбалансированной многопутевой сортировки (Рисунок 21):

- Время — 252.233 мс.
- Количество сравнений — 94361.
- Количество перестановок — 128328.

```
===== Laboratory Word #2 =====
Goal: Comparative analysis of external memory sorting algorithmes efficiency.
Level 'Good': Direct merge sorting, balanced multiway merge sorting.

============= MENU =============
1. Sort a single data file
2. Statistical data collection mode
3. Close the program
Choose execution mode: 1
Enter the size of the numeric sequence: 15000
Enter the minimum value of an element: 1
Enter the maximum value of an element: 10000
File "data.txt" with random data has been generated.

Choose the external sorting method:
1. Direct merge sort
2. Balanced multiway merge sort
Your choice: 2
Enter the number of ways (m) for merging (e.g., 3): 3

Sorting results:
Execution time: 252.233 ms
Number of comparisons: 94361
Number of swaps (writes): 128238
Sorted data is in the file "data.txt".
```

Рисунок 21 — Режим 1. Сбалансированная многопутевая сортировка

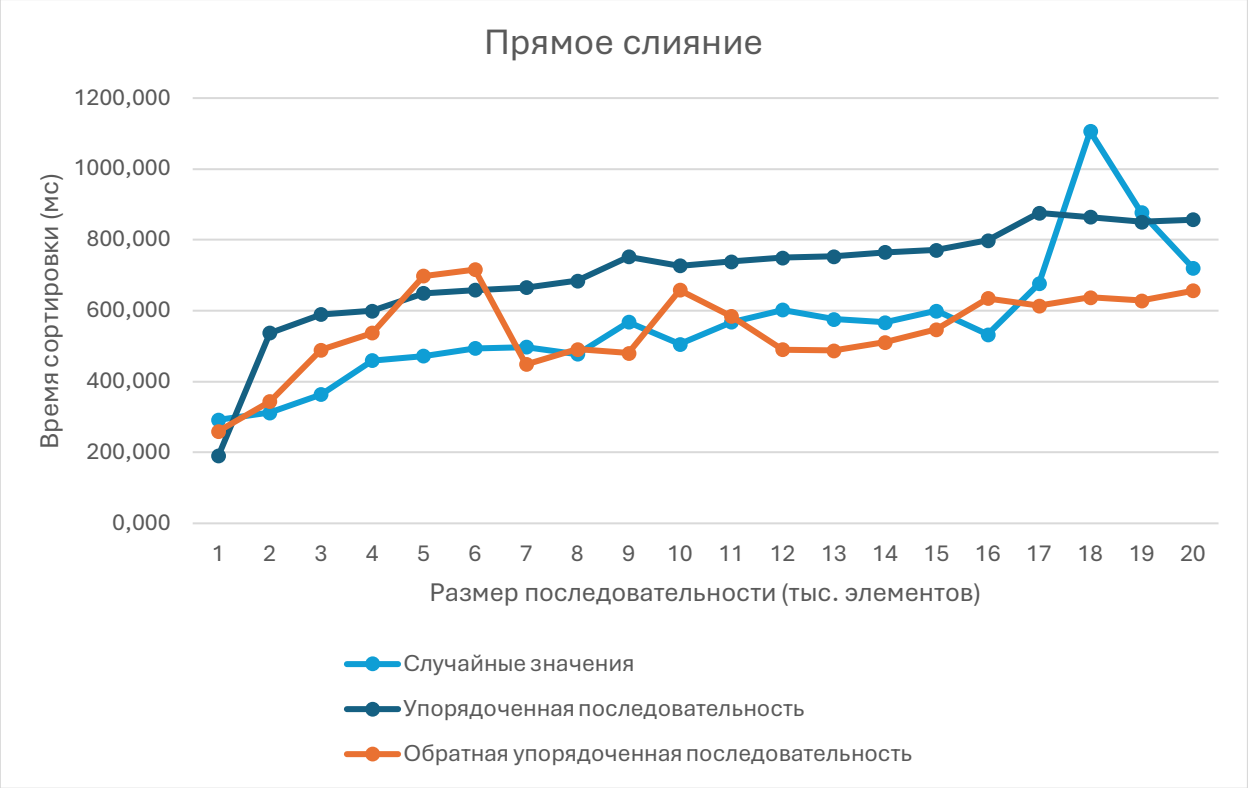
Для точности результатов вычисления были проделаны ещё 4 раза, по итогам которых результаты отличались не более чем на 15% процентов для обоих способов сортировки.

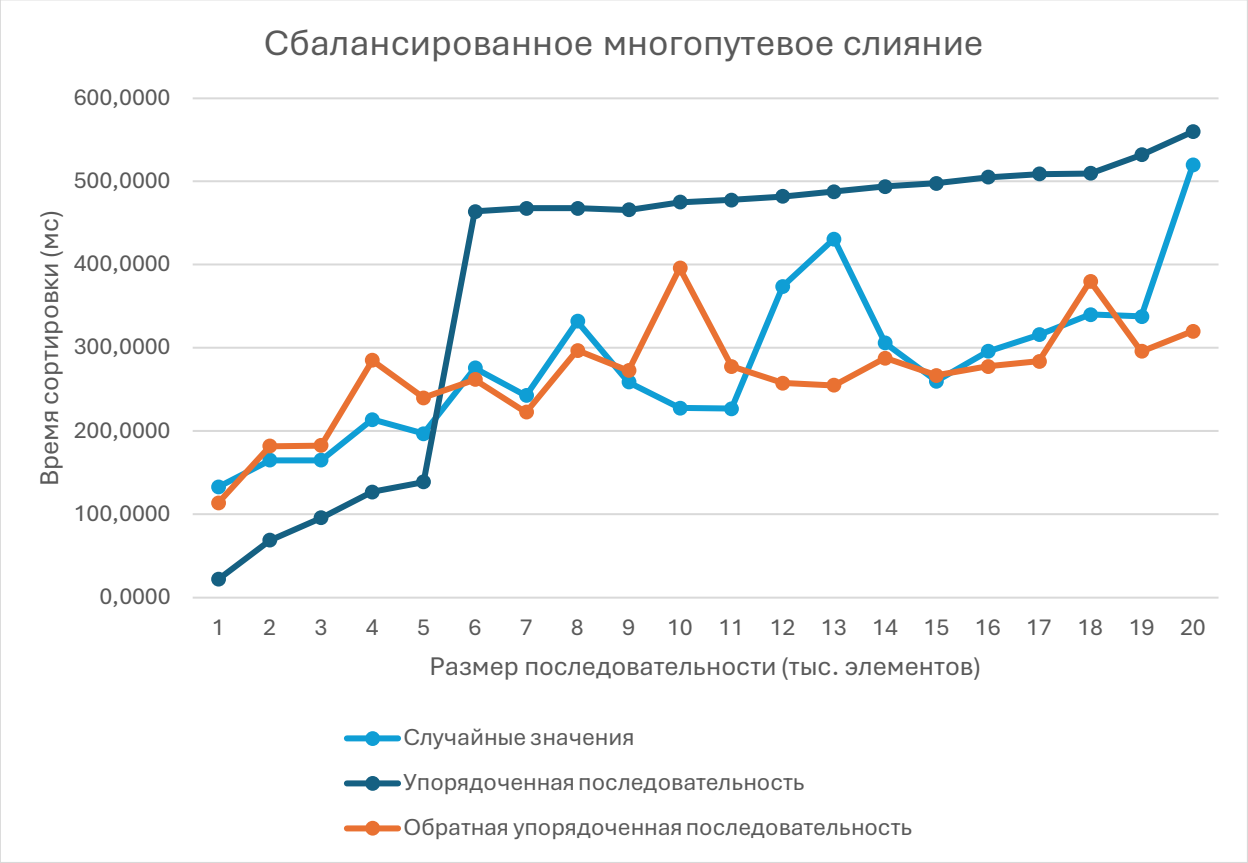
Для сравнения методов сортировки во втором режиме были выбраны следующие параметры:

- начальный размер последовательности — 1000;
- конечный размер последовательности — 20000;
- шаг изменения последовательности — 1000.

Данные параметры являются оптимальными, так как выполнение программы не требует большого количества времени, а полученные результаты удобны для расположения на графике для демонстрации результатов.

Ниже представлены графики, иллюстрирующие результаты работы методов сортировки при разных способах заполнения последовательности.





По итогам тестирования методов сортировки было выяснено, что сортировка методом сбалансированного многопутевого слияния при использованных параметрах выполняется в среднем на 93% быстрее, чем при использовании метода прямого слияния.

Кроме того, оба метода выполняются в среднем быстрее при сортировке обратной упорядоченной последовательности, чем при упорядоченной (в среднем на 27%) и при последовательности случайных значений (в среднем на 4%). Исходя из этих показателей, можно сделать вывод о том, что данные методы сортировки не являются естественными, так как изменение времени сортировки при наличии частично упорядоченных последовательностей незначительно.

ВЫВОДЫ

В ходе лабораторной работы были изучены методы сортировки данных во внешней памяти, особенности их программной реализации и эффективности работы на различных наборах исходных данных.

Метод сортировки сбалансированным многопутевым слиянием оказался эффективнее метода сортировки прямым слиянием в среднем в 2 раза, что подтверждается разницей во времени выполнения сортировки, а также количеством сравнений и перестановок.

На основании разницы времени выполнения методов сортировки в зависимости от способа формирования последовательности можно сделать вывод о том, что данные методы сортировки не являются естественными.